

Кеширование в веб-разработке

Кочетов
Даниил

- Что такое кеширование
- Цепочка кешей в HTTP-запросе
- Кеш в Bitrix, Symfony
- Кеш в Next.js
- Стратегии, инвалидация
- Ошибки, мониторинг
- Современные тренды

Что такое кеширование

Запрос → кеш → источник → вычисления → сохранение в кеш → ответ

// Добавить схему

Тут пример

веб/практик

Ключевые термины

- Hit rate / miss rate
- TTL (Time To Live)
- Тёплый / холодный кеш
- Stale data
- Invalidation

Цепочка кешей (HTTP Flow)

Пользователь → DNS-кеш → CDN → Браузер (HTTP, localStorage, SW) →
→ Reverse Proxy (Nginx, Varnish) → Веб-сервер → Приложение (Bitrix, Symfony, Next.js) →
→ Distributed Cache (Redis, Memcached) → База данных

// Добавить схему

Уровни кеширования

- L1/L2/L3 кеш процессора
- Page cache, buffer cache
- Дискový кеш (SSD/HDD)

- Кеш IP-адресов
- Локальный и облачный резолвер
- Ускорение разрешения имён

- Кеширование статики (JS, CSS, images)
- Cloudflare, AWS CloudFront, Яндекс.CDN
- Снижение RTT, защита от DDoS

Браузерный кеш

- HTTP-кеш (Cache-Control, ETag)
- localStorage, sessionStorage, IndexedDB
- Service Workers (PWA)

Reverse Proxy

- Кеширование HTTP-ответов
- Nginx, Varnish
- Разгрузка бэкенда

Кеш на уровне приложения

Bitrix: типы кеша

- Управляемый кеш (Managed Cache)
- Неуправляемый кеш (CACHE_TIME)
- Тегированный кеш (Tagged Cache)
- Композитный кеш
- Кеш шаблонов, настроек, ORM
- Внешний кеш (Redis, Memcached)

```
$cache = \Bitrix\Main\Data\Cache::createInstance();

if ($cache->initCache(3600, 'catalog_data', '/catalog/')) {
    $data = $cache->getVars();
} elseif ($cache->startDataCache()) {
    $data = getCatalogData();
    $cache->endDataCache($data);
}
```

Bitrix: Тегированный кеш

```
$cache = \Bitrix\Main\Data\Cache::createInstance();  
  
if ($cache->startDataCache(3600, 'catalog_items_5', '/catalog/', ['iblock_id_5'])) {  
    $items = \CIBlockElement::GetList([], ['IBLOCK_ID' => 5]);  
    $cache->endDataCache($items);  
} else {  
    $items = $cache->getVars();  
}  
  
// Инвалидация  
  
\Bitrix\Main\Data\TaggedCache::clearByTag('iblock_id_5');
```


Symfony: типы кеша

- Кеш конфигов, роутов, сервисов
- HTTP-кеш (Cache-Control, ETag)
- Doctrine L2 Cache (query, result)
- Tag-aware Cache
- Adapters: Redis, Memcached

Symfony: Tag-aware Cache

```
$cache->get('products_list', function (ItemInterface $item) {  
    $item->tag(['products', 'category_5']);  
    return $this->fetchProducts();  
});
```

// Инвалидация:

```
$cache->invalidateTags(['category_5']);
```

Кеш с тегами

Next.js: типы кеша

- SSG — статическая генерация
- ISR — инкрементальная регенерация
- Edge Functions (на CDN)
- API Routes + Cache-Control
- Клиентский кеш: React Query, SWR

```
export async function getStaticProps() {  
  const posts = await fetchPosts();  
  return {  
    props: { posts },  
    revalidate: 60 // обновление каждую минуту  
  };  
}
```

Incremental Static Regeneration

```
export const runtime = 'edge';

export async function GET() {
  return new Response('Hello from Edge', {
    headers: { 'Cache-Control': 's-maxage=60' }
  });
}
```

Выполнение на CDN

Стратегии кеширования

Cache-aside (Lazy Loading)

- Приложение проверяет кеш
- При промахе — читает из источника
- Сохраняет в кеш

- Приложение запрашивает данные
- Кеш сам читает из источника при промахе
- Кеш кэширует ответ

Write-through

- Запись одновременно в кеш и источник
- Гарантия согласованности
- Высокая нагрузка на запись

Write-behind (Write-back)

- Запись только в кеш
- Асинхронная запись в источник
- Риск потери данных

Refresh-ahead (Prefetch)

- Обновление кеша до истечения TTL
- Снижение latency
- Требуется прогнозирования

Cache warming

- Предзаполнение кеша при старте
- Для критичных данных
- Снижает нагрузку на холодный старт

Инвалидация

Инвалидация

- По времени (TTL)
- По событию (data change)
- Tag-based (зависимости)
- Ручная (manual)

- Cache stampede
- Hot keys
- Thundering herd
- Memory leak
- Stale data

Мониторинг и безопасность

Метрики

- Hit/miss rate
- Latency, throughput
- Memory usage, eviction rate

- Изоляция кеша (multi-tenant)
- Cache poisoning
- Минимум PII в кеше
- Шифрование чувствительных данных

Когда НЕ кешировать

- Часто меняющиеся данные
- Персонализированный контент
- Критичные по актуальности
- Редко запрашиваемые данные

- Edge Computing (Cloudflare Workers)
- ML-driven кеширование
- Serverless кеширование
- Адаптивные TTL

Вопросы